StudyMind v2 — Implementation Plan

Oct 25, 2026 · @Daddy

Goal

StudyMind v2 turns the current Cognee-backed notebook app into an **adaptive study agent**: it retrieves from your material with its own multimodal RAG pipeline, quizzes you, and remembers where you are weak. The resume claim it should earn is one you can defend line by line in an interview, backed by measured numbers.

The upgrade rests on three ideas, each a core AI-engineering topic:

- **Own the retrieval.** Build chunking, embeddings, hybrid search and reranking yourself, keep Cognee as a baseline to beat.
- **Make it an agent.** A LangGraph state machine that plans, calls tools, checks its own answers and loops.
- **Prove it.** An evaluation harness with a fixed question set, retrieval and faithfulness metrics, and ablations.

Target resume bullet (numbers filled in from your eval in Phase 6):

> Built StudyMind, an adaptive study agent (LangGraph, Qdrant, FastAPI): multimodal hybrid RAG over PDFs, slides and lectures with page-level citations, plus a learner-memory layer that targets weak topics with spaced repetition. Raised Recall@5 from X% to Y% over the managed baseline and cut unsupported claims by Z% via a self-check node, measured on a 150-question eval set.

What the current code does, and where it falls short

The app is cleanly organised, but every intelligent step is one `POST /api/v1/recall` call to Cognee Cloud, and several calls misuse it. These are the issues to fix, each mapped to the phase that fixes it.

Issue	Where	Why it matters	Fixed in
Instructions are sent as the retrieval query. The study-guide, audio-script and topic-suggestion prompts go into `query`, so the vector search embeds "Create a study guide… Structure the guide exactly as follows…" instead of the topic.	`core/topics.py` `_llm_call`, `generate_study_guide`, `suggest_topic_name`	Retrieval quality collapses: the embedding matches formatting words, not content. Classic query/instruction separation bug.	Phase 2
Cross-topic synthesis re-retrieves globally. The final prompt (with chunks pasted in) is sent as a new `GRAPH_COMPLETION` query with no `datasets` filter.	`core/query.py` `cross_topic_ask` step 3	The carefully scoped chunks are mixed with a fresh, unscoped search; topic scoping is lost.	Phase 2
Chunks are truncated to 300 characters before synthesis.	`cross_topic_ask`	Answers are built on sentence fragments.	Phase 2
Fallback only fires on HTTP error, not on empty results.	`ask`	Empty retrieval returns "No results" instead of trying another strategy.	Phase 3 (agent retry)

Issue	Where	Why it matters	Fixed in
"Entity overlap" is exact string equality of whole lines.	`core/graph_utils.py` `_extract_entity_names`	Two topics almost never share an identical line, so Option A in `connect_topics` effectively never fires.	Phase 2 (entity extraction)
No citations. Page markers like `[Page 3]` are inlined into one text blob; sources come back as dataset names only.	`core/ingest.py` `ingest_pdf`, `query.py` `_extract_sources_from_recall`	Answers cannot be verified; no faithfulness evaluation is possible.	Phase 1
Text only. PyMuPDF `get_text()` drops diagrams, tables and equations.	`ingest_pdf`	Study material is visual; diagram questions are unanswerable.	Phase 4
`async def` functions call blocking `httpx.post`.	`ingest.py`, `query.py`	Event loop blocks; cross-topic loops run serially.	Phase 0
"Forget" does not delete anything in the cloud.	`forget_source`, `forget_all`	Privacy gap: users think data is gone.	Phase 1
`topics.json` is the database; every function re-reads and re-writes it.	`core/topics.py`	Race conditions, no multi-user support.	Phase 0
`pyttsx3` is used but missing from `requirements.txt`; `check_ollama` aliases are dead code.	`requirements.txt`, `core/config.py`	Fresh installs break audio; leftover code signals an unfinished migration.	Phase 0

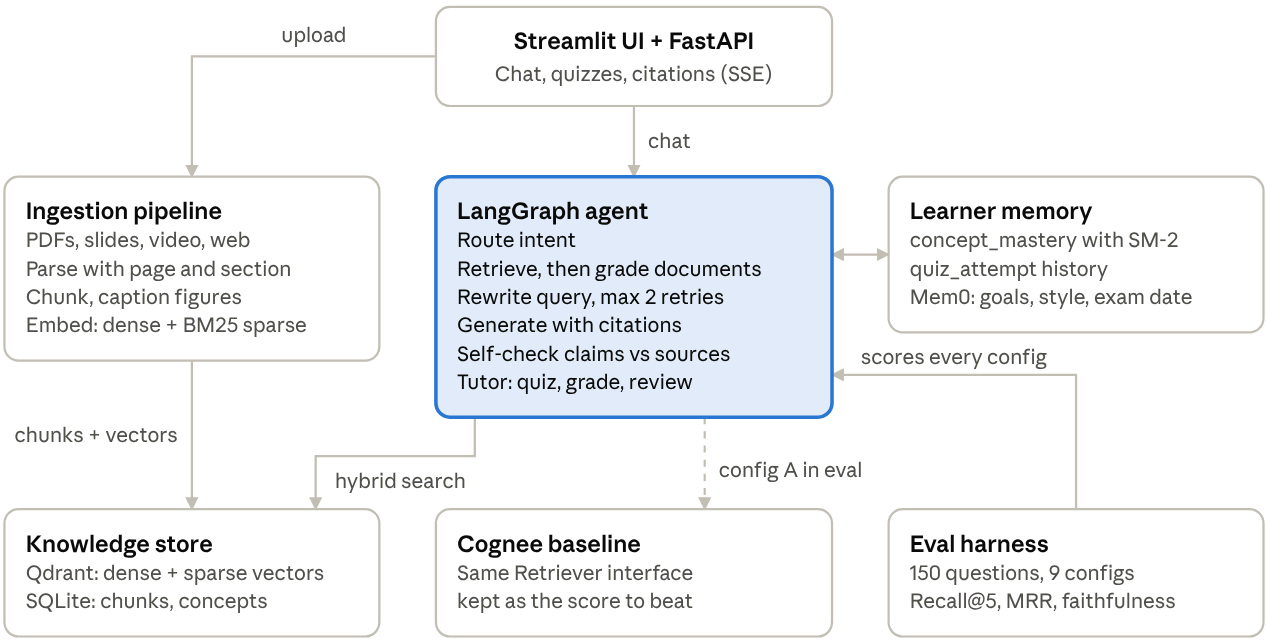
Issue	Where	Why it matters	Fixed in
Tests are health checks and helpers only.	`tests/test_ingest.py`	Nothing measures answer quality.	Phase 6

Keep what works: the topic/notebook model, the four ingestion sources, study-guide caching with invalidation, and the Streamlit UX ideas.

Target architecture

Two kinds of memory, one agent: knowledge about the material lives in Qdrant and SQLite, knowledge about you lives in learner memory, and only the LangGraph agent touches both.

One agent sits between your material and your learner memory



StudyMind v2 architecture · 6 components

The Cognee baseline sits behind the same `Retriever` interface as your hybrid retriever, so the eval can swap one for the other. Langfuse traces every node and tool call (not drawn).

Phased plan

Eight phases, each ending in something that runs and is committed. Every phase names the theory it puts into practice, so the project doubles as your study curriculum.

Phase 0 — Foundation refactor (week 1)

Split the app into a backend you can test and a thin UI, before adding any AI.

Concepts: separation of concerns, async I/O, provider abstraction, observability.

- [ ] Move logic behind a FastAPI service (`api/`); Streamlit only calls HTTP endpoints.
- [ ] Replace `topics.json` with SQLite via SQLModel: tables `topic` , `source` , `chunk` , `quiz_attempt` , `concept_mastery` .
- [ ] Switch to `httpx.AsyncClient` ; run per-topic retrievals with `asyncio.gather` .
- [ ] Add an `LLMClient` wrapper (LiteLLM or the OpenAI SDK) so models are one config line; log tokens, cost and latency per call.
- [ ] Add tracing with Langfuse (or LangSmith) from day one; every later phase is debugged through it.
- [ ] Clean up: settings via `pydantic-settings` , add `pyttsx3` to requirements, delete `check_ollama` aliases.

Done when: the existing features work through the API, and `pytest` passes without network access.

Phase 1 — Own ingestion and hybrid retrieval (weeks 2–3)

Build the RAG core yourself, behind a `Retriever` interface with two implementations: `CogneeRetriever` (today's behaviour, kept as the baseline) and `HybridRetriever` (yours).

Concepts: chunking trade-offs, bi-encoders, BM25, hybrid search, Reciprocal Rank Fusion, cross-encoder reranking, metadata filtering.

- [ ] Parse with structure: PyMuPDF blocks or Docling, keeping `source_id` , `page` , `section_heading` on every chunk.
- [ ] Implement two chunkers to compare later: fixed 512 tokens with 64 overlap, and structure-aware (split on headings, then by size).
- [ ] Embed with one open model (e.g. `bge-small-en-v1.5`) and one API model (`text-embedding-3-small`); store in Qdrant with payload filters on `topic_id` .

- [ ] Add sparse BM25 retrieval (Qdrant sparse vectors or `rank_bm25`) and fuse with dense results via RRF (k = 60).
- [ ] Rerank the fused top 30 down to 6 with a cross-encoder (`bge-reranker-base` or a hosted reranker).
- [ ] Return citations as `{source_title, page, chunk_id}` and render them as clickable chips in the UI.
- [ ] Make delete real: removing a source deletes its Qdrant points and DB rows.

Done when: scoped Q&A answers cite page numbers, and a deleted source never appears in results again.

Phase 2 — Query understanding and grounded generation (week 4)

Fix the instruction-as-query bug and make every generation feature grounded and structured.

Concepts: query/instruction separation, query rewriting, HyDE, decomposition, structured outputs, map-reduce summarisation, LLM-based entity extraction.

- [ ] Every feature sends a short *retrieval query* and a separate *generation prompt*; never one string for both.
- [ ] Add query rewriting: multi-query (3 paraphrases) and HyDE (embed a hypothetical answer); keep both behind flags for ablation.
- [ ] Cross-topic questions: decompose into sub-questions, retrieve per topic, synthesise once with the scoped chunks only.
- [ ] Study guide: map-reduce over all chunks of a topic (summarise per section, then merge), returned as a Pydantic `StudyGuide` model.
- [ ] Generation prompt requires an inline citation per claim, and an explicit "not in your material" when context is missing.
- [ ] Topic connections: extract `(concept, relation, concept)` triples per chunk with an LLM at ingest time; store in a `concept_edge` table; compute overlap on normalised concept names.

Done when: study guides cover every section of a topic, and the topic map shows real shared concepts.

Phase 3 — The LangGraph agent (weeks 5–6)

Replace the fixed tab-per-feature pipeline with one agent that decides what to do. This is the step that makes it an agent, not an app.

Concepts: ReAct tool use, state machines, Corrective RAG (grade retrieved documents, rewrite and retry), Self-RAG (check the answer against its sources), loop limits, checkpointing.

- [ ] Define `StudyState` (TypedDict): `messages`, `topic_id`, `intent`, `query`, `documents`, `answer`, `citations`, `retries`, `learner_profile`.
- [ ] Nodes: `route_intent` (qa / explain / quiz / review / connect), `retrieve`, `grade_documents`, `rewrite_query`, `generate`, `self_check`, `tutor`.
- [ ] Tools the agent can call: `search_material`, `generate_quiz`, `grade_answer`, `get_weak_concepts`, `schedule_review`.
- [ ] Conditional edges: low document relevance → `rewrite_query` (max 2 retries) → still nothing → say so honestly.
- [ ] `self_check` flags claims not supported by cited chunks; unsupported answer → regenerate once, then return with a warning.
- [ ] Persist conversations with a SQLite checkpointer, keyed by `thread_id`.
- [ ] Stream node events to the UI so the user sees "searching → checking sources → answering".

Done when: a vague question triggers a visible rewrite-and-retry in the trace, and the self-check catches a planted unsupported claim in a unit test.

Phase 4 — Multimodal material (week 7)

Make diagrams, tables and lecture videos searchable and citable.

Concepts: caption-then-embed, multimodal and late-interaction retrieval (CLIP, ColPali), table extraction, timestamped transcripts.

- [ ] Extract figures and tables per page (PyMuPDF `get_images`, Docling tables); store image files with `page` and bounding box.
- [ ] Caption each figure with a vision LLM ("describe this diagram for a student; list labels and relationships"); embed the caption as a chunk with `modality = figure` and a link to the image.
- [ ] Store tables as Markdown chunks so numbers stay retrievable.
- [ ] When a figure chunk is cited, show the image inline next to the answer.
- [ ] YouTube: keep transcript timestamps; cite as `video @ 12:34` with a deep link.
- [ ] Stretch: a `ColPaliRetriever` over page screenshots, compared against caption-then-embed in Phase 6.

Done when: a question answerable only from a diagram returns the right figure and page.

Phase 5 — Learner memory and adaptive tutoring (week 8)

Add the second memory: what the system knows about *you*, not about the material.

Concepts: semantic vs episodic memory, memory extraction and update (Mem0's ADD/UPDATE/DELETE), spaced repetition (SM-2), knowledge tracing, LLM-as-judge grading with rubrics.

- [ ] Tag every chunk and quiz question with 1–3 concepts from the Phase 2 concept table.
- [ ] Quiz generation: MCQ and short-answer from retrieved chunks, each with a citation and a difficulty label, as Pydantic models.
- [ ] Grade short answers with a rubric prompt (correct / partially correct / wrong + one-line feedback); log every attempt in `quiz_attempt`.
- [ ] Update `concept_mastery` per attempt with SM-2 (`ease`, `interval_days`, `next_review`).
- [ ] Add Mem0 for free-form learner facts (goals, exam date, preferred explanation style), injected into the `tutor` node.
- [ ] "Review" intent: the agent pulls due and weakest concepts, quizzes on them, and explains mistakes from the material.

Done when: after a session with planted wrong answers, the next "review" session prioritises exactly those concepts.

Phase 6 — Evaluation harness (weeks 9–10)

Turn every design choice above into a measured comparison. The next section specifies the dataset and metrics.

Concepts: golden datasets, retrieval metrics (Recall@k, MRR), RAG metrics (faithfulness, answer relevance, context precision), LLM-as-judge and its biases, ablation studies.

- [ ] Build the eval set in `eval/dataset.jsonl` and a runner `python -m eval.run --config <name>` that writes `eval/results/<name>.json`.
- [ ] Implement Recall@5 and MRR directly from `gold_chunk_ids`; use RAGAS (or your own judge prompts) for faithfulness and relevance.
- [ ] Validate the judge: hand-label 30 answers, report agreement with the judge before trusting it.

- [ ] Run every row of the experiments table below; commit the results table to the README.
- [ ] Add a 20-question smoke eval to CI that fails if Recall@5 drops more than 5 points.

Done when: the README has a results table with real numbers and one chart.

Phase 7 — Production polish (weeks 11–12)

Make it something a recruiter can open and use in two minutes.

Concepts: streaming, caching, prompt-injection defence for ingested content, cost control, containerisation, CI/CD.

- [ ] `docker compose up` starts `api`, `qdrant` and `ui`; one `.env` file.
- [ ] Stream tokens over SSE; cache embeddings by content hash; cache repeated queries.
- [ ] Treat ingested text as data: wrap chunks in delimiters, instruct the model never to follow instructions inside them, and test with a planted "ignore previous instructions" PDF.
- [ ] Per-user cost cap and rate limit on the API.
- [ ] GitHub Actions: lint, unit tests, smoke eval.
- [ ] Deploy a demo (Hugging Face Spaces, Render or Fly.io) with a sample course pre-loaded.
- [ ] README: architecture diagram, 60-second demo GIF, results table, "design decisions" section.

Evaluation plan

The eval is 150 hand-written questions over one real course you are taking, so every number on your resume comes from material you can show.

Dataset. Pick one course with lecture PDFs, slides and at least two lecture videos. Each row of `eval/dataset.jsonl` holds `question`, `type`, `gold_answer`, `gold_chunk_ids`, `topic_id`.

Question type	Count	Example	What it tests
Text fact	50	"What is the time complexity of heap insert?"	Basic retrieval

Question type	Count	Example	What it tests
Diagram or table only	30	"In the TCP state diagram, what follows FIN_WAIT_1?"	Phase 4 multimodal
Multi-hop, one topic	25	Needs two chunks from different pages	Reranking, query rewriting
Cross-topic	20	Connects two notebooks	Decomposition
Unanswerable	25	Plausible but not in the material	Honest refusal, self-check

Metrics.

- Retrieval: Recall@5 and MRR against `gold_chunk_ids`.
- Generation: faithfulness (share of claims supported by cited chunks), answer correctness vs `gold_answer`, citation accuracy.
- Behaviour: refusal rate on unanswerable questions, mean latency, cost per question.
- Tutoring (Phase 5): simulate a learner who answers 3 planted concepts wrong; measure the share of review questions that target those concepts.

Experiments. Change one variable per row against the previous best, so each gain is attributable.

Config	Change	Recall@5	Faithfulness	Refusal on unanswerable	Latency (s)	Cost / q ($)
A	Cognee baseline (current app)					
B	Dense only, fixed 512-token chunks					
C	+ structure-aware chunking					
D	+ BM25 hybrid with RRF					
E	+ cross-encoder reranker					
F	+ query rewriting (multi-query / HyDE)					
G	+ figure captions (multimodal)					
H	+ LangGraph grade-and-retry + self-check					

Config	Change	Recall@5	Faithfulness	Refusal on unanswerable	Latency (s)	Cost / q ($)
I	Stretch: ColPali page retrieval vs G					

Report the diagram-only questions separately: the G-vs-F gap on them is your multimodal headline. Report H's refusal rate on unanswerable questions: that is your hallucination headline.

Repo structure after the upgrade

One folder per responsibility, so each phase touches a small, known set of files.

```
StudyMind/
├── api/                    # FastAPI app: routes for topics, ingest, chat
(SSE), quiz
├── studymind/
│   ├── config.py           # pydantic-settings
│   ├── db.py               # SQLModel tables: topic, source, chunk,
concept_edge,
│   │                       #   quiz_attempt, concept_mastery
│   ├── llm.py              # LLMClient: model routing, token/cost logging
│   ├── ingest/             # parsers (pdf, youtube, web, note), chunkers,
figure captioner
│   ├── retrieval/          # Retriever interface, CogneeRetriever,
HybridRetriever,
│   │                       #   reranker, query rewriting
│   ├── agent/              # LangGraph: state.py, nodes.py, tools.py,
graph.py
│   ├── tutor/              # quiz generation, rubric grading, SM-2
scheduler, Mem0 wrapper
│   └── prompts/            # versioned prompt templates
├── ui/                     # Streamlit app calling the API
├── eval/                   # dataset.jsonl, run.py, metrics.py, results/
├── tests/                  # unit tests per module; no network needed
├── docker-compose.yml      # api, qdrant, ui
└── .github/workflows/ci.yml
```

Your current files map across as: `core/ingest.py` → `studymind/ingest/`, `core/query.py` → `retrieval/` and `agent/`, `core/topics.py` → `db.py` and `tutor/`, `core/graph_utils.py` → `ui/` (rendering) plus `concept_edge` queries, `app.py` → `ui/`.

Resume bullets and interview talking points

Use two or three bullets, each pairing one thing you built with one number from Phase 6. Replace X, Y, Z only with measured values.

- Built **StudyMind**, an adaptive study agent (LangGraph, FastAPI, Qdrant) over PDFs, slides and lecture videos, with page- and timestamp-level citations.
- Designed a hybrid retrieval pipeline (BM25 + dense, RRF fusion, cross-encoder reranking, structure-aware chunking), raising Recall@5 from X% to Y% over a managed graph-RAG baseline on a 150-question eval set.
- Added multimodal retrieval via vision-LLM figure captioning, improving accuracy on diagram-only questions from X% to Y%.
- Implemented a corrective agent loop (document grading, query rewriting, self-check), cutting unsupported claims by Z% and correctly refusing N% of unanswerable questions.
- Built a learner-memory layer (Mem0 + SM-2 spaced repetition) that targets weak concepts, steering X% of review questions to planted weak areas in simulation.

Be ready to explain these in an interview; each maps to a phase:

Likely question	Your answer draws on
Why hybrid search instead of only embeddings?	BM25 catches exact terms (formulas, acronyms); dense catches paraphrase; RRF merges ranks without score calibration. Show rows C–D.
Why rerank, and why not rerank everything?	Cross-encoders read query and chunk together, so they are accurate but slow; run them on the top 30 only. Show row E plus latency.
How do you stop hallucinations?	Grounding prompt with required citations, `self_check` node, honest refusal path; measured by faithfulness and the unanswerable set.
How do you know your LLM judge is right?	Hand-labelled 30 answers and reported agreement before trusting it.

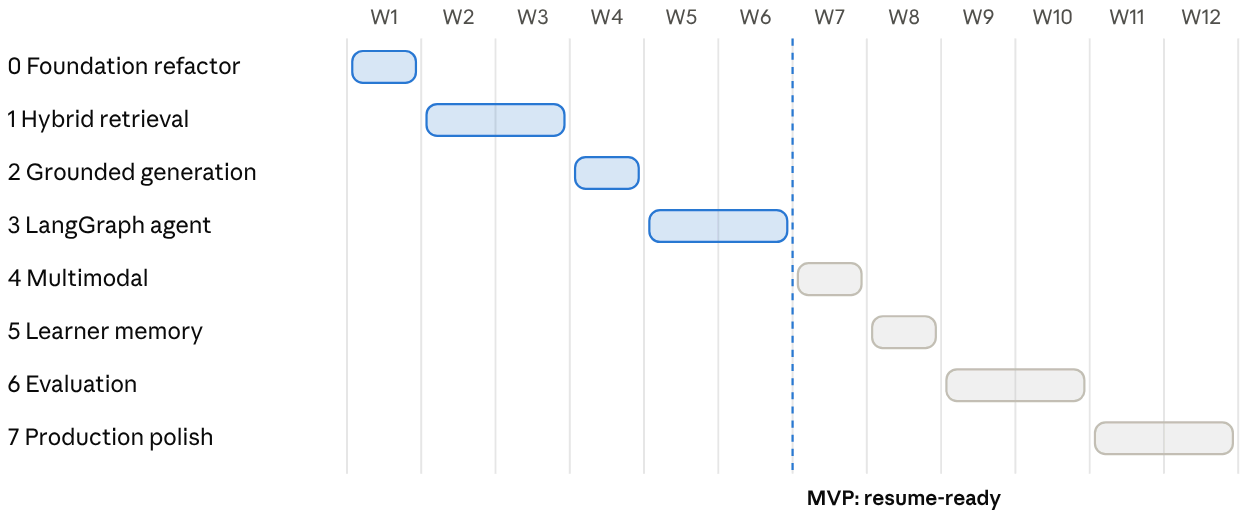
Likely question	Your answer draws on
What was the biggest bug you found?	Instructions sent as the retrieval query in v1; explain why embeddings matched formatting words, and the fix.
Agent vs pipeline: when is an agent worth it?	Only where the next step depends on results (retry, quiz flow); show row H's gain against its extra latency and cost.
How does memory differ from RAG?	RAG stores knowledge about the material; learner memory stores facts and performance about the user, with update and decay rules.

Timeline, risks and scope cuts

Twelve weeks of part-time work, with a usable resume version at week 6: Phases 0–3 plus a 30-question eval run on configs A, B and E.

A resume-ready MVP lands at week 6, the full upgrade at week 12

Phases by week; shaded phases make up the MVP



StudyMind v2 roadmap · 8 phases over 12 weeks

Start writing eval questions in week 2, about 15 a week, so Phase 6 is mostly running experiments rather than labelling.

Risk	Fallback
Scope creep across eight phases	Ship the week-6 MVP first, put it on your resume, then extend.
Labelling gold chunks is slow	Label `gold_chunk_ids` with a helper script that shows the top 10 retrieved chunks and lets you tick the right ones.
API cost of captioning and LLM-as-judge	Use a small model for both, cache by content hash, run the full eval only at milestones.
Cognee Cloud limits or API changes	Record baseline results in week 3 and commit them, so config A never needs rerunning.
ColPali needs a GPU	Keep it as a stretch; run it on a free Colab GPU or skip it.
LangChain and LangGraph API churn	Pin versions in `requirements.txt` and follow the current official docs, not older tutorials.

If you fall behind, cut in this order: ColPali, the audio overview, Mem0 (keep SM-2 in SQLite), the public deploy (keep Docker plus a demo GIF). Never cut the eval: it is what makes the project credible.